第 13 章練習解答　I/O 系統

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS13-E01　PIO 發布與遺失喚醒

1. CPU 設 READY=1；controller 看到 READY，令 BUSY=1 並讀舊 DATA；CPU 才寫新 DATA。故新資料並未參與該命令，完成通知也不能修復。必須以裝置規範要求的 accessor/barrier 使資料先可見，再發布命令。
2. waiter 讀 false；IRQ 寫 true 並 wake（此時尚無 waiter）；waiter 入睡。以同一 lock 保護 predicate 與 wait queue：while !done 原子入隊並釋放 lock 睡眠；IRQ 在 lock 下設 done 並喚醒。while 同時處理其他原因的喚醒。

檢查。 不得把 IRQ 次數當成資料完成狀態。

OS13-E02　部分向量寫入

1. 第一次先消耗120，再消耗第二段50，剩 [base1+50,30]、[base2,300]，總330。第二次消耗30，再消耗第三段100，剩 [base2+100,200]。已送300，未送200。
2. 不應。保持已送300與剩餘200，等待可寫或依上層取消規則處理。重新從原向量送出會重複300 bytes；EAGAIN 不代表先前成功資料被撤銷。

檢查。 總長500=已送300+剩餘200。

OS13-E03　虛擬 timer

1. 初始設7。插入D成[6D,7A,10B,10C]，重設6；取消A後仍設6。到6取D，重設10；到10一次取B/C。若計時器回報延遲，處理所有deadline≤now，不只等於now的一項。
2. 否。hardware/IRQ 延遲、deferred work 與排程都可延後 callback；deadline 是觸發要求，不是無條件的 user 執行保證。

檢查。 同時到期的 B/C 都須處理，不能只彈出一筆。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 13 章練習解答　I/O 系統 （續）

OS13-E04　DMA 取消與資源回收

1. 裝置可能仍寫入原 dma_addr；頁面若被重配給另一行程就可能跨行程毀損。只有確知裝置停止存取之後，才可同步、unmap、釋放。
2. state 在共同 lock/CAS 下由 IN_FLIGHT 轉 DONE 或 ABORTED；只有成功轉換者有發布與解除DMA buffer映射資格。任意兩事件若都宣稱成功，第二次必須從已terminal狀態再次成功轉移，與轉移前提矛盾。這是 safety，不保證故障裝置最終回覆。
3. 不能。晚到abort/timer/IRQ callback可能仍讀state；各callback須先持存活reference，退出才release。例owner/IRQ/abort三引用：IRQ完成後3→2，owner退出2→1，abort讀DONE後退出1→0才free。buffer硬體已不存取與control block沒有software讀者是兩個不同條件。

檢查。 使用者返回timeout、DMA buffer可解除映射及control block可回收是三個時點。

OS13-E05　有限批次 double buffer

1. A1 in0–3/process3–8；B2 in3–6/process8–13；A3 in8–11/process13–18；B4 in13–16/process18–23。兩槽讓輸入/計算重疊，但裝置需等槽空。
2. 單buffer4×(3+5)=32 ms；雙buffer3+5+3×5=23 ms。穩態每5 ms一塊、CPU瓶頸。有限批次不能直接算4×5=20，還有最初3 ms填滿。

檢查。 各buffer的輸入與處理不重疊；不同buffer才可重疊。

OS13-E06　STREAMS 高低水位

1. 接受後900，達高水位停止。取一筆變600仍未低於低水位；再取一筆300，低於400，back-enable。此模型的低水位比較明定為小於；不要自行改成另一個實作的≤。
2. 不能。下游300加上游600至少900 B；backpressure只是分散等待位置。若外部資料源不能停止且所有queue滿，仍須drop/節流，不能推出無限容量。

檢查。 stream head 的阻塞與 driver 收包時不可無限等待分開。

OS13-E07　多資源上限與 Little 關係

1. device60000/s；CPU1/(10×10^{-6})=100000/s；memory2×2^{30}/(24×2^{10})=87381.33/s。上界min=60000/s，device瓶頸。
2. L=50000×0.0004=20；Ls=50000×0.0001=5；Lq=15，Wq=0.0003 s。必須用同一穩態工作負載與觀測邊界。
3. 不能。平均每秒至少積欠10000筆，queue無界增長；有限穩態前提失敗，不能套固定L=λW估成穩定20筆。

檢查。 GiB=2^{30} B；吞吐資源界不保證實際可達。

第 13 章練習解答　I/O 系統 （續）

OS13-E08　命名、LBA 與 durability

1. 12345=3×4096+57，故block內位移57；partition LBA90×8=720，整碟起始LBA1744。本次bytes57–156位於該block內；若整blockcache fill會讀1744–1751共8sectors，不能把要求100 B當成硬體必讀100 B。
2. 不能，只表示該API已接受100 B。dirty page仍可能只在RAM，controller volatile cache也可能未落媒體；須透過檔案系統同步與裝置flush/FUA等相容契約建立耐久保證。
3. 不能。若printer不保存可查詢去重狀態，同job id也可能重新列印；主機無法只憑未收到ack分辨未印與已印。須額外裝置協定，否則標示結果未知。

檢查。 命名/快取/裝置/持久化是不同邊界。

原創驗算程式附錄: io_models.py

```
"""Finite examples, not proofs of real driver correctness."""
from itertools import permutations

def consume(lengths, n):
    if n<0 or n>sum(lengths):raise ValueError('invalid result length')
    for i,length in enumerate(lengths):
        if n<length:return i,n,[length-n]+lengths[i+1:]
        n-=length
    return len(lengths),0,[]

def double_buffer(n, tin, tcpu):
    slots=[0,0];device=cpu=0;trace=[]
    for i in range(n):
        slot=i%2;begin=max(device,slots[slot]);end=begin+tin;device=end
        p=max(cpu,end);cpu=p+tcpu;slots[slot]=cpu;trace.append((begin,end,p,cpu))
    return trace

def terminal(events):
    state='IN_FLIGHT';publish=0;freed=False
    for event in events:
        if event=='timeout':continue # not evidence of quiescence
        if state=='IN_FLIGHT':
            state='DONE' if event=='completion' else 'ABORTED';publish+=1;freed=True
    return state,publish,freed

def control_lifetime():
    refs={'owner','IRQ','abort'}
    trace=[len(refs)]
    for actor in ['IRQ','owner','abort']:
        assert actor in refs # callback owns a reference before accessing state
        refs.remove(actor);trace.append(len(refs))
    return trace
```

第 13 章練習解答　I/O 系統 （續）

```
class PIO:
    def __init__(self):self.busy=0;self.ready=0;self.data=None;self.delivered=[]
    def submit(self,value):
        if self.busy or self.ready:return False
        self.data=value;self.ready=1;return True
    def accept(self):
        assert self.ready and not self.busy
        self.busy=1;self.delivered.append(self.data)
    def complete(self):
        assert self.busy
        self.ready=self.busy=0

if __name__=='__main__':
    p=PIO();assert p.submit(0x41)
    assert p.busy==0 and p.ready==1
    assert not p.submit(0x42) and p.data==0x41
    p.accept();assert not p.submit(0x42)
    p.complete();assert p.submit(0x42)
    p.accept();p.complete();assert p.delivered==[0x41,0x42]
    print('PIO two submissions: READY=1/BUSY=0 prevents overwrite; delivered [0x41,0x42]')
    assert consume([120,80,300],170)==(1,50,[30,300])
    assert consume([30,300],130)==(1,100,[200])
    t=double_buffer(3,4,6);assert t==[(0,4,4,10),(4,8,10,16),(10,14,16,22)]
    assert double_buffer(4,3,5)[-1][-1]==23
    assert terminal(['timeout'])==('IN_FLIGHT',0,False)
    assert control_lifetime()==[3,2,1,0]
    for order in permutations(['timeout','completion','abort_ack']):
        assert terminal(order)[1:]==(1,True)
    print('iovec remainders:',consume([120,80,300],170),consume([30,300],130))
    print('double-buffer trace:',t)
    print('6 terminal-event orders: at most one publication; timeout alone keeps DMA alive')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 13 章練習解答　I/O 系統 （續）

原創驗算程式附錄: io_timeline.py

```
"""Compose queue, device, and completion latency.

Run:
    python io_timeline.py
"""

from dataclasses import dataclass


@dataclass(frozen=True)
class Request:
    queued_at: int
    device_ns: int


def simulate(requests: list[Request]) -> list[tuple[int, int]]:
    now = 0
    result: list[tuple[int, int]] = []
    for request in requests:
        start = max(now, request.queued_at)
        finish = start + request.device_ns
        result.append((start, finish))
        now = finish
    return result


def main() -> None:
    timeline = simulate([Request(0, 5), Request(2, 3), Request(20, 4)])
    assert timeline == [(0, 5), (5, 8), (20, 24)]
    total_wait = sum(start - req.queued_at for (start, _), req in zip(timeline, [Request(0,
5), Request(2, 3), Request(20, 4)]))
    assert total_wait == 3
    print("timeline", timeline, "queue_wait", total_wait)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study